# Statistical Programming Languages

## Time and date handling & classical times series analysis

Katarzyna Reluga[1]

[1]Acknowledgements: Thanks to Manuel Pfeuffer, Maarten Jung, and Tobias Wistub for sharing notes from earlier editions.

# Current time

- R provides different object classes for handling time points
  - `POSIXct` is used if time zones and daylight saving is important
  - `POSIXlt` as before just uses list components

```
# format: year-month-day hour:minute:second timezone
Sys.time() # get the current time
## [1] "2025-09-17 22:45:22 CEST"
# format: year-month-day
Sys.Date() # get the current date
## [1] "2025-09-17"
```

- Time zones
  - CET = Central European Time (UTC+1)
  - UTC = Coordinated Universal Time

# Time point representation

- Standard byte formats cannot span very long periods while maintaining precision.
- Time points are measured (in seconds or days) from a reference time point (epoch), often 1970/01/01 00:00:00

```
# internal storage (seconds) since 1970/01/01 00:00:00
as.numeric(Sys.time())
## [1] 1758141922
# convert a numerical value to a time point
as.POSIXct(1581319917, origin="1970/01/01 00:00:00")
## [1] "2020-02-10 08:31:57 CET"
```

- Year 2000 problem: years stored with two digits; after 1999-12-31 some systems rolled to 1900-01-01.
- Year 2038 problem: with signed 32-bit seconds since 1970-01-01 00:00:00 UTC, values overflow at 2038-01-19 03:14:07 UTC, so later times cannot be represented.

# Date conversions

```
as.POSIXct(Sys.Date(), origin="1970/01/01 00:00:00")
## [1] "2025-09-17 UTC"
as.Date(Sys.Date()) # Only date, without UTC
## [1] "2025-09-17"
# German date format
as.Date("1.1.2020", format="%d.%m.%Y")
## [1] "2020-01-01"
```

| Code | Value |
|---|---|
| %d | Day of the month (decimal number) |
| %m | Month (decimal number) |
| %b | Month (abbreviated) |
| %B | Month (full name) |
| %y | Year (2 digit) |
| %Y | Year (4 digit) |

# Time conversions

```
as.POSIXct(Sys.time(), origin="1970/01/01 00:00:00")
## [1] "2025-09-17 22:45:22 CEST"
# German format and time first, date last
as.POSIXct("10:15 3.4.20",
           origin="1970/01/01 00:00:00",
           format="%H:%M %m.%d.%Y")
## [1] "0020-03-04 10:15:00 LMT"
```

| Code | Value |
|---|---|
| %H | Hour (00–23) |
| %M | Minute (00–59) |
| %S | Second (00-61) |

# Conversion codes I

| Code | Value |
|---|---|
| %a | Abbreviated weekday name in the current locale on this platform |
| %A | Full weekday name in the current locale |
| %b | Abbreviated month name in the current locale on this platform |
| %B | Full month name in the current locale |
| %c | Date and time. Locale-specific on output, "%a %b %e %H:%M:%S %Y" on input |
| %C | Century (00–99): the integer part of the year divided by 100 |
| %d | Day of the month as decimal number (01–31). |
| %D | Date format such as %m/%d/%y |
| %e | Day of the month as decimal number (1–31), with a leading space for a single-digit number. |
| %F | Equivalent to %Y-%m-%d (the ISO 8601 date format). |
| %g | The last two digits of the week-based year (see %V). (Accepted but ignored on input.) |
| %G | The week-based year (see %V) as a decimal number. (Accepted but ignored on input.) |

# Conversion codes II

| Code | Value |
|---|---|
| %h | Equivalent to %b |
| %H | Hours as decimal number (00–23), 24:00:00 are accepted for input |
| %I | Hours as decimal number (01–12) |
| %j | Day of year as decimal number (001–366) |
| %m | Month as decimal number (01–12) |
| %M | Minute as decimal number (00–59) |
| %n | Newline on output, arbitrary white space on input. |
| %p | AM/PM indicator in the locale |
| %r | For output, the 12-hour clock time (using the locale's AM or PM). For input, equivalent to %I:%M:%S %p |
| %R | Equivalent to %H:%M |
| %S | Second as integer (00–61), |
| %t | Tab on output, arbitrary white space on input |
| %T | Equivalent to %H:%M:%S |

# Conversion codes III

| Code | Value |
|---|---|
| %u | Weekday as a decimal number (1–7, Monday is 1) |
| %U | Week of the year as decimal number (00–53) using Sunday as the first day 1 of the week |
| %V | Week of the year as decimal number (01–53) as defined in ISO 8601 |
| %w | Weekday as decimal number (0–6, Sunday is 0) |
| %W | Week of the year as decimal number (00–53) using Monday as the first day of week |
| %x | Date. Locale-specific on output, "%y/%m/%d" on input |
| %X | Time. Locale-specific on output, "%H:%M:%S" on input |
| %y | Year without century (00–99). On input, values 00 to 68 are prefixed by 20 and 69 to 99 by 19 |
| %Y | Year with century. For input, only years 0:9999 are accepted |
| %z | Signed offset in hours and minutes from UTC, so -0800 is 8 hours behind UTC |

# Extract from timepoints

```
t <- as.POSIXct("2020/04/03 10:15:00",
                origin="1970/00/00 00:00:00")
weekdays(t, abbreviate=TRUE)
## [1] "Fri"
months(t, abbreviate=TRUE)
## [1] "Apr"
quarters(t)
## [1] "Q2"
xl <- as.POSIXlt(t) # uses list components
# sec, min, hour, mday, mon, year, wday,
# yday, isdst, zone, gmtoff
xl$year # year since 1900
## [1] 120
xl$zone # time zone
## [1] "CEST"
```

# Add time to time points & time point sequences

```
t <- as.POSIXct("2020/04/03 10:15:00",
                origin="1970/00/00 00:00:00")
t
## [1] "2020-04-03 10:15:00 CEST"
t + 3600 # add 3600 seconds
## [1] "2020-04-03 11:15:00 CEST"
t <- as.Date("2020/04/03")
t + 30   # add 30 days
## [1] "2020-05-03"
seq(as.Date("2020/04/03"), as.Date("2020/04/09"),
    by="3 days")
## [1] "2020-04-03" "2020-04-06" "2020-04-09"
```

# Time point differences

```
t1 <- as.POSIXct("2020/04/03 10:15:00",
                 origin="1970/00/00 00:00:00")
t2 <- as.POSIXct("2020/04/03 13:45:00",
                 origin="1970/00/00 00:00:00")
t2-t1
## Time difference of 3.5 hours
difftime(t2, t1)
## Time difference of 3.5 hours
as.numeric(difftime(t2, t1))
## [1] 3.5
difftime(t2, t1, units="secs")
## Time difference of 12600 secs
as.numeric(difftime(t2, t1, units="secs"))
## [1] 12600
```

# Package lubridate - Introduction

- lubridate makes it easier/possible to do things in R with date-times
- lubridate cheat sheet
  - how to round dates, work with time zones, extract elements of a date or time, parse dates
  - lubridate's three time span classes: periods, durations, and intervals and explains how to do math with date-times

```
library("lubridate")
##
## Attaching package: 'lubridate'
## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

# Package lubridate - Creation

- Current time and data

```
now()
## [1] "2025-09-17 22:45:22 CEST"
today()
## [1] "2025-09-17"
```

- **Parse date-times**
  - Identify the component order in your data: year (y), month (m), day (d); hour (h), minute (m), second (s).
  - Use the function whose name mirrors that order, e.g. `ymd_hms()`, `dmy()`, `mdy_hm()`.

```
ymd_hms("2020/04/03 10:15:00")
## [1] "2020-04-03 10:15:00 UTC"
```

# Package lubridate - Extraction

```
t <- as.POSIXct("2020/04/03 10:15:00",
                origin="1970/00/00 00:00:00")
date(t)
## [1] "2020-04-03"
week(t)
## [1] 14
day(t) <- 9 # Replace day to 9
t
## [1] "2020-04-09 10:15:00 CEST"
```

- date, year, isoyear, epiyear, month, day, wday, qday
- hour, minute, second, week, isoweek, epiweek, quarter
- semester, am, pm, dst, leap_year

# Package lubridate - Time spans

- lubridate provides three classes of time spans: periods: period, durations: duration, and intervals: %within%

```
# add a 14-day DURATION (exact 14*86400 seconds)
ymd("2020-01-01") + ddays(14)
## [1] "2020-01-15"
# a PERIOD of 3 months and 12 days (calendar-aware span)
months(3) + days(12)
## [1] "3m 12d 0H 0M 0S"
# a DURATION object: exactly 14 days in seconds
ddays(14)                # ignores DST/clock changes
## [1] "1209600s (~2 weeks)"
# interval from 2020-01-01 to 2020-04-03
interval(ymd("2020-01-01"), ymd("2020-04-03"))
## [1] 2020-01-01 UTC--2020-04-03 UTC
# same as above; %--% is shorthand for interval()
ymd("2020-01-01") %--% ymd("2020-04-03")
## [1] 2020-01-01 UTC--2020-04-03 UTC
```

# Time series - Creation

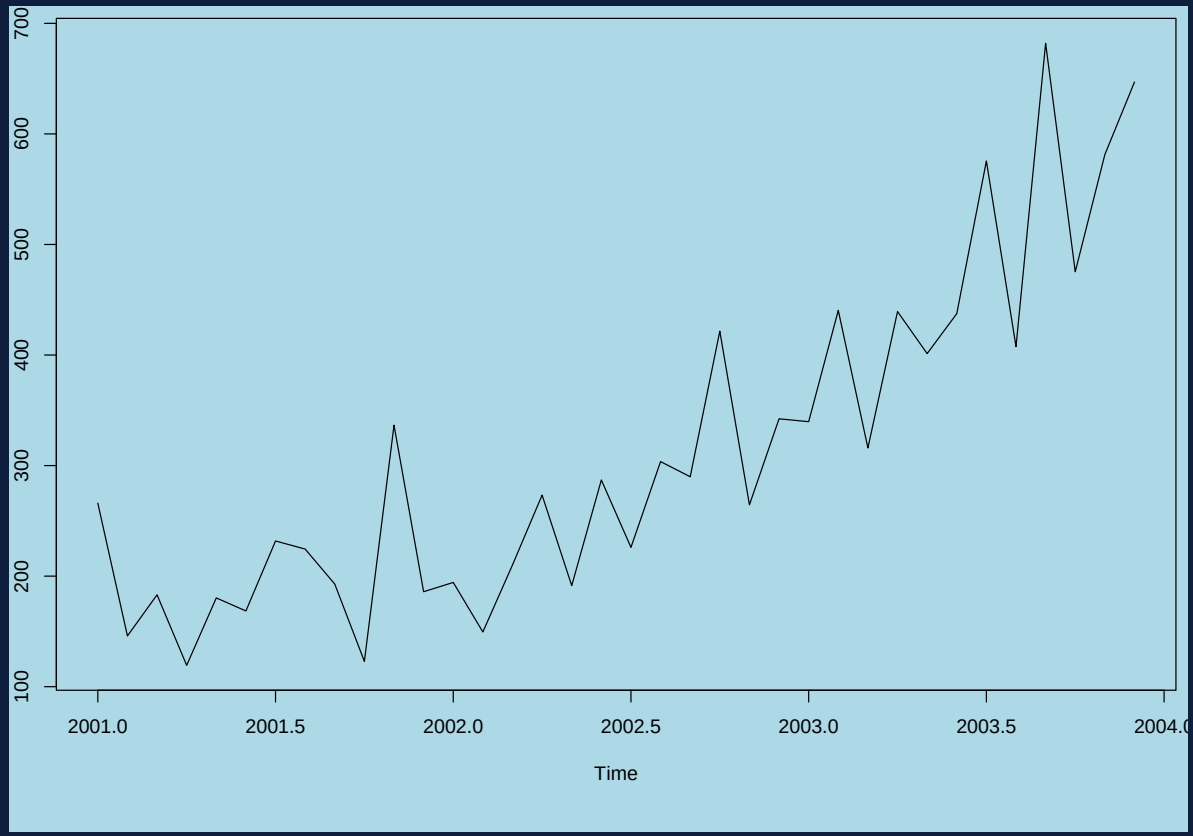
```
# Shampoo sales
url <- paste0("https://raw.githubusercontent.com/",
              "jbrownlee/Datasets/master/shampoo.csv")
library("rio")
xt <- import(url)
head(xt[,2], 3)
## [1] 266.0 145.9 183.1
# ts time series object
t  <- ts(xt[,2], start=c(2001, 1), frequency=12)
```

```
t
##       Jan   Feb   Mar   Apr   May   Jun   Jul   Aug   Sep   Oct   Nov   Dec
## 2001 266.0 145.9 183.1 119.3 180.3 168.5 231.8 224.5 192.8 122.9 336.5 185.9
## 2002 194.3 149.5 210.1 273.3 191.4 287.0 226.0 303.6 289.9 421.6 264.5 342.3
## 2003 339.7 440.4 315.9 439.3 401.3 437.4 575.5 407.6 682.0 475.3 581.3 646.9
```

```
summary(t)
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   119.3   192.4   280.1   312.6   411.1   682.0
```

# Time series - Plot

```
par(bg = "lightblue", mar = c(6, 2, 0.5, 0.5))
plot(t)
```



# Time series - Trend & Seasonality I

- Time series $(t, x_t) \quad t = 1, \ldots, T$ can be decomposed in
  - an increasing or decreasing trend (systematic),
  - a repeating short-term cycle (systematic), and
  - a random variation/noise (unsystematic)
- Trend functions
  - geometric mean $\hat{x}_t = x_1 i^{t-1}$ with $i = \sqrt[T-1]{x_T/x_1}$
  - linear trend $\hat{x}_t = at + b$
  - exponential trend $\hat{x}_t = exp(b)\exp(a)^t \Leftrightarrow log(x_t) = at + b$
  - moving average $\hat{x}_t = \sum_j w_j x_{t+j}$ with $\sum_j w_j = 1, j \in \mathbb{Z}$
- Seasonality with sub periods $p = 1, \ldots, P$,
  e.g. $P = 4$ (quarterly), $P = 12$ (monthly)
  - If $t$ falls in sub period $p$
    - additive $\hat{x}_{t,p} = \hat{x}_t + \hat{s}_p$ with $\hat{s}_p = \text{mean}(x_t - \hat{x}_t)$
    - multiplicative $\hat{x}_{t,p} = \hat{x}_t \hat{s}_p$ with $\hat{s}_p = \text{mean}(\frac{x_t}{\hat{x}_t})$

# Time series - Trend & Seasonality II

- Model quarterly data ($p = 1, 2, 3, 4$) with multiplicative seasonality

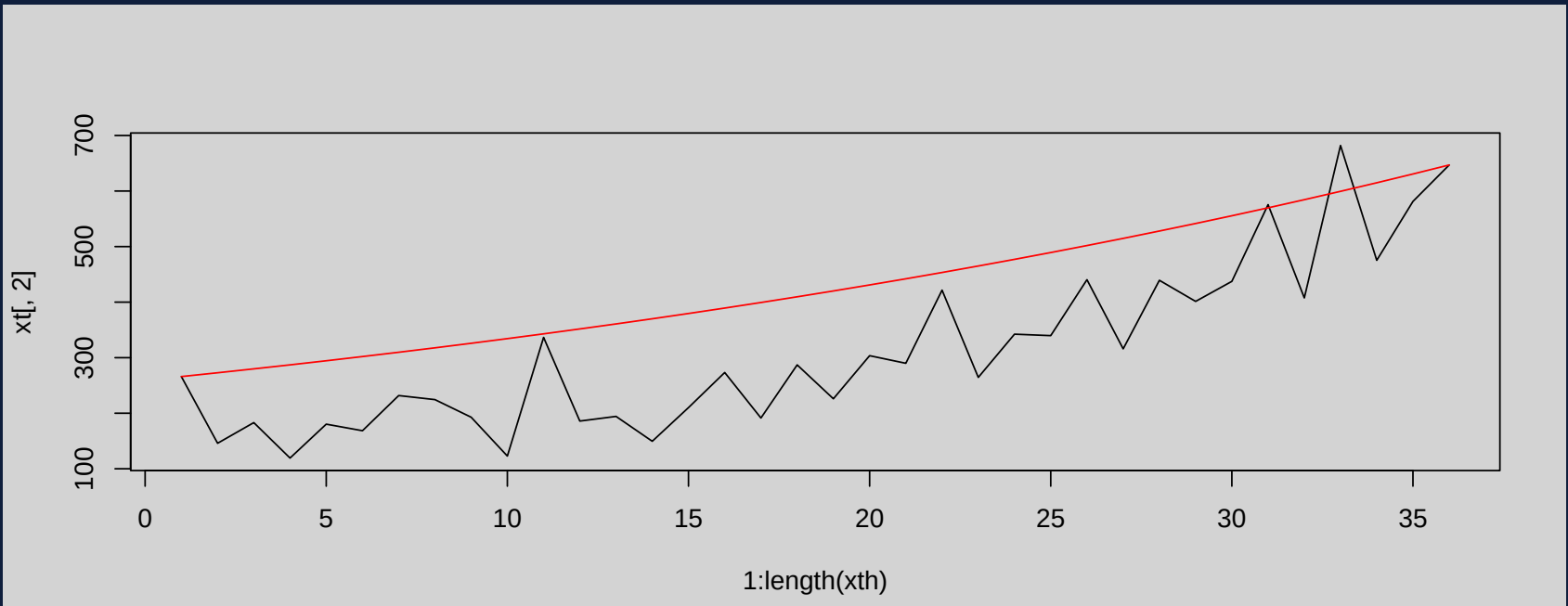
| $x_t$ | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $x_6$ | $x_7$ | $x_8$ | $x_9$ | $x_{10}$ | $x_{11}$ | $x_{12}$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\hat{x}_t$ | $\hat{x}_1$ | $\hat{x}_2$ | $\hat{x}_3$ | $\hat{x}_4$ | $\hat{x}_5$ | $\hat{x}_6$ | $\hat{x}_7$ | $\hat{x}_8$ | $\hat{x}_9$ | $\hat{x}_{10}$ | $\hat{x}_{11}$ | $\hat{x}_{12}$ |
| $p$ | 1 | 2 | 3 | 4 | 1 | 2 | 3 | 4 | 1 | 2 | 3 | 4 |
| $\hat{s}_1$ | $\frac{x_1}{\hat{x}_1}$ | | | | $\frac{x_5}{\hat{x}_5}$ | | | | $\frac{x_9}{\hat{x}_9}$ | | | |
| $\hat{s}_2$ | | $\frac{x_2}{\hat{x}_2}$ | | | | $\frac{x_6}{\hat{x}_6}$ | | | | $\frac{x_{10}}{\hat{x}_{10}}$ | | |
| $\hat{s}_3$ | | | $\frac{x_3}{\hat{x}_3}$ | | | | $\frac{x_7}{\hat{x}_7}$ | | | | $\frac{x_{11}}{\hat{x}_{11}}$ | |
| $\hat{s}_4$ | | | | $\frac{x_4}{\hat{x}_4}$ | | | | $\frac{x_8}{\hat{x}_8}$ | | | | $\frac{x_{12}}{\hat{x}_{12}}$ |
| $\hat{x}_{t,p}$ | $\hat{x}_1\hat{s}_1$ | $\hat{x}_2\hat{s}_2$ | $\hat{x}_3\hat{s}_3$ | $\hat{x}_4\hat{s}_4$ | $\hat{x}_5\hat{s}_1$ | $\hat{x}_6\hat{s}_2$ | $\hat{x}_7\hat{s}_3$ | $\hat{x}_8\hat{s}_4$ | $\hat{x}_9\hat{s}_1$ | $\hat{x}_{10}\hat{s}_2$ | $\hat{x}_{11}\hat{s}_3$ | $\hat{x}_{12}\hat{s}_4$ |

- Model evaluation

$$R^2 = 1 - \frac{\sum(x_t - \hat{x}_{t,p})^2}{\sum(x_t - \text{mean}(x_t))^2}$$

# Time series - Geometric mean

```r
# compute rates
n    <- nrow(xt)
rate <- (xt[n,2]/xt[1,2])^(1/(n-1)) # compute rate value
rate
## [1] 1.025716
xth  <- xt[1,2]*rate^(0:(n-1)) #estimate geometric model
par(bg = "lightgrey")
plot(1:length(xth), xt[,2], type="l")
lines(1:length(xth), xth, col="red")
```



# Time series - Linear trend

```r
tv <- as.numeric(t) #coerce time series to numeric
df <- data.frame(t=1:length(tv), xt=tv) #create a data fram
# Linear trend
lt <- lm(xt~t, data=df) #fit linear regression
```

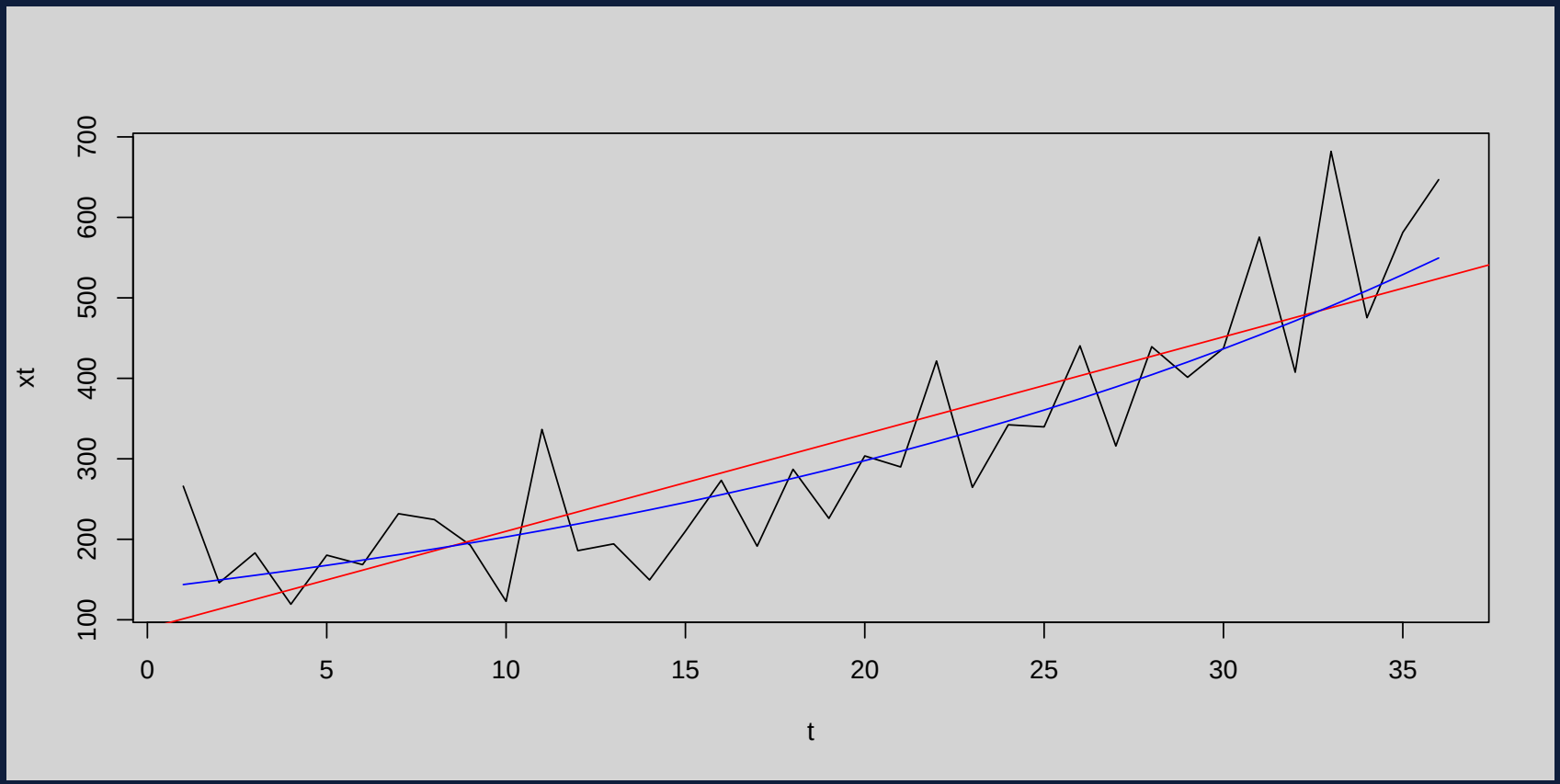
```r
summary(lt)
##
## Call:
## lm(formula = xt ~ t, data = df)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -108.74  -52.12  -16.13   43.48  194.25
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)    89.14      26.72   3.336  0.00207 **
## t              12.08       1.26   9.590 3.37e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 78.5 on 34 degrees of freedom
## Multiple R-squared:  0.7301, Adjusted R-squared:  0.7222
## F-statistic: 91.97 on 1 and 34 DF,  p-value: 3.368e-11
```

# Time series - Exponential trend

```
# Exponential trend
# $\hat{x}_t=exp(b)\exp(a)^t --> log(x_t)=at+b$
et <- lm(log(xt)~t, data=df)
et
##
## Call:
## lm(formula = log(xt) ~ t, data = df)
##
## Coefficients:
## (Intercept)            t
##     4.93002      0.03831
# R^2
1-sum((df$xt-exp(fitted(et)))^2)/(nrow(df)*var(df$xt))
## [1] 0.7988027
```

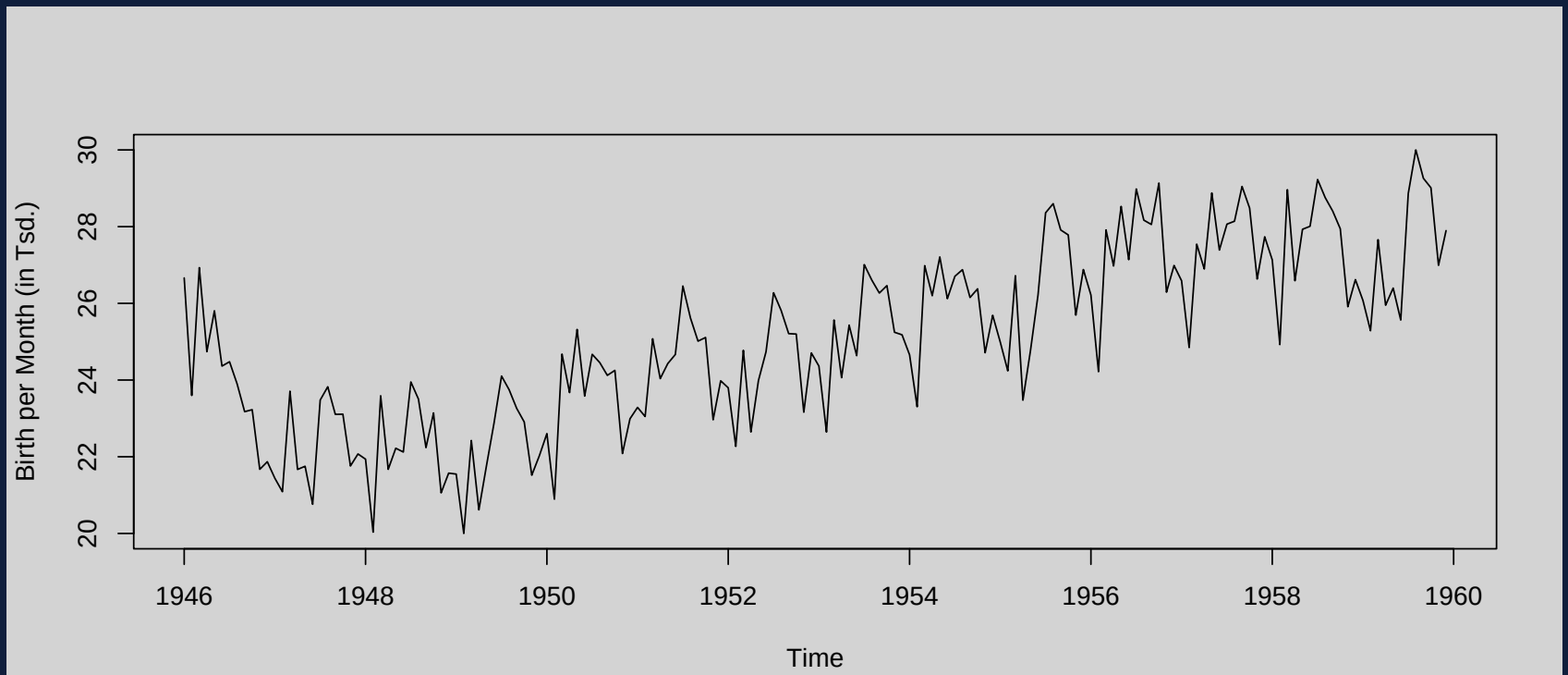
# Time series - Plotting trends

```
# Plot both trends
par(bg = "lightgrey")
plot(df, type="l")
abline(lt, col="red") #linear
lines(df$t, exp(fitted(et)), col="blue") #exponential
```



# Time series - Another time series

```
# Number of births per month in New York city,
# from January 1946 to December 1958.
url <- paste0("https://robjhyndman.com/tsdldata/",
              "data/nybirths.dat")
xt <- import(url, format = "csv")
t  <- ts(xt, start=c(1946, 1), frequency=12)
par(bg = "lightgrey")
plot(t, ylab="Birth per Month (in Tsd.)")
```



# Time series - Decompose

```
# Trend: moving average with additive seasonality
dt <- decompose(t)
str(dt)
## List of 6
##  $ x       : Time-Series [1:168, 1] from 1946 to 1960: 26.7 23.6 26.9 24.7 25.8 ...
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : NULL
##   .. ..$ : chr "V1"
##  $ seasonal: Time-Series [1:168] from 1946 to 1960: -0.677 -2.083 0.863 -0.802 0.252 ...
##  $ trend   : Time-Series [1:168, 1] from 1946 to 1960: NA NA NA NA NA ...
##  $ random  : Time-Series [1:168, 1] from 1946 to 1960: NA NA NA NA NA ...
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : NULL
##   .. ..$ : chr "x - seasonal"
##  $ figure  : num [1:12] -0.677 -2.083 0.863 -0.802 0.252 ...
##  $ type    : chr "additive"
##  - attr(*, "class")= chr "decomposed.ts"
# Trend: moving average with multiplicative seasonality
dtm <- decompose(t, type = "multiplicative")
```

# Time series - Plot time series components

```
par(bg = "lightgrey")
plot(dt)
```